A Customized AI Chatbot for Educational Environments: Integrating Retrieval-Augmented Generation with Curriculum-Aligned Responses

Thuan N. A. Trang[1], Phat Tran[2], Tuan M. Le[1], Ly Le[1], Greeni Maheshwari[1], Nhu Do[1], Son V. T. Dao[1*]
[1]School of Science, Engineering & Technology, RMIT University, Vietnam
[2]School of Computing Australian National University, Canberra, Australia
Email: s3698989@rmit.edu.vn, u7565296@anu.edu.au, s4116946@student.rmit.edu.au, ly.le9@rmit.edu.vn, greeni.maheshwari@rmit.edu.vn, nhu.dothihuong@rmit.edu.vn, son.daovutruong@rmit.edu.vn
Corresponding authors: Son V. T. Dao (son.daovutruong@rmit.edu.vn)

2025 RIVF International Conference on Computing and Communication Technologies (RIVF) | 979-8-3315-7790-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/RIVF68649.2025.11365126

***Abstract*— We present a customized Artificial Intelligence (AI) chatbot tailored for educational environments. Unlike conventional models that rely solely on pretrained knowledge, our system integrates rule-based keyword matching and retrieval-augmented generation (RAG) with internal educational resources to produce accurate, contextually relevant responses aligned with specific lesson plans. The chatbot adapts answers based on user attributes such as age group, predefined lesson schemas, and institution-defined content. To improve precision, it actively engages users by requesting clarification when needed. We deploy a compact large language model, Qwen-14B, optimized to run efficiently on a local setup with 16GB VRAM. This requires a sophisticated workflow to sustain performance and responsiveness. For general queries beyond the internal database, the chatbot seamlessly switches to a general-purpose mode, balancing flexibility with educational alignment.**

Keywords — Educational Chatbot, Retrieval-Augmented Generation, Rule-Based Filtering, Curriculum Alignment

I. Introduction

Advances in large language models (LLMs) have enabled powerful tools for natural language processing, retrieval, and human-computer interaction [1], [2]. Chatbots such as ChatGPT, Copilot, Gemini, Perplexity, and Grok offer instant responses to diverse queries [3], [4], [5]. Despite their versatility, these systems often lack alignment with lesson plans, risk bypassing learning processes, and remain costly for institutions [6], [7], [8].

We propose an educational chatbot integrating rule-based keyword filters with retrieval-augmented generation (RAG) over curated resources, ensuring accuracy, curricular alignment, and adaptive responses based on learner attributes. The system seeks clarification for vague queries and supports differentiated instruction. To avoid commercial LLM costs, we adopt Qwen-14B [9], deployable on local hardware with 16GB VRAM. Through workflow design—task decomposition, edge-case handling, prompt tuning—and a queuing mechanism for concurrent users, the system achieves scalable, low-latency performance.

II. Related works

Large Language Models in Education LLMs are increasingly used in education for tutoring, answering queries, and administrative support [10], [11]. Commercial systems like ChatGPT, Gemini, Copilot, Perplexity, and Grok provide general conversational capabilities but often lack curricular alignment and may produce pedagogically inconsistent or factually incorrect responses. Limitations include hallucination [12], [13], outdated knowledge [14], and limited transparency or personalization. Retrieval-Augmented Generation (RAG) [15] addresses these by integrating external knowledge retrieval, improving accuracy, adaptability, and curriculum alignment.

Retrieval-Augmented Generation for Education RAG combines retrieval with generation to deliver domain-specific, accurate, and interpretable outputs [16]. It has proven effective in curriculum - aligned QA systems across subjects [17] and in MOOCs [18]. Educational chatbots like Moodle-Bot [10], ChatEd [19], and EduChat [19] extend RAG to multi-turn dialogues with instructional strategies, while systems like VizChat [20] integrate multimodal analytics. These approaches enhance engagement, personalization, and learning quality.

Open-Source Models for Privacy, Cost, and Adaptability Closed-source LLMs offer high performance but raise privacy and cost concerns. Open-source models such as LLaMA, Qwen, and DeepSeek allow local deployment, ensuring data control, cost savings, and fine- tuning flexibility [4], [9], [21]. Toolkits like Unsloth [22] enable efficient parameter-tuning (LoRA, QLoRA) for educational use.

Our Approach We integrate open-source LLMs with three components: (1) rule-based safety and curriculum filters co-designed with educators, (2) a RAG module retrieving from lesson-aligned knowledge bases, and (3) lightweight personalization adapting responses to student profiles. This unified, modular pipeline ensures privacy, interpretability, and pedagogical alignment while supporting on-premises deployment with resource-efficient serving backends.

III. Background

Retrieval-Augmented Generation Retrieval-Augmented Generation (RAG) extends large language models (LLMs) by incorporating external knowledge retrieval during inference. While LLMs such as GPT-3 have demonstrated impressive generative capabilities, their dependence on static pretraining data limits their ability to provide up-to-date or domain-specific information. RAG addresses this limitation by retrieving relevant context from external sources before generating responses, thereby improving factuality, adaptability, and reliability.

Core Components A standard RAG system comprises the following stages:

- Retrieval: Given a query, the system searches external knowledge sources (e.g., document stores, APIs, vector databases) to retrieve relevant passages.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:46:29 UTC from IEEE Xplore. Restrictions apply.

- Augmentation: Retrieved content is filtered, summarized, or structured to provide coherent context aligned with the query.
- Generation: The language model synthesizes the retrieved context with its internal knowledge to generate accurate and fluent responses.

System Enhancements To improve performance in real-world educational and enterprise environments, we introduce the following enhancements:

- Hybrid Retrieval: By combining sparse retrieval methods (e.g., BM25) with dense embedding-based retrievers (e.g., DPR), our system captures both exact term matches and semantic relevance.

- Contextual Re-Ranking: Retrieved passages are re-ranked using neural scoring models to prioritize the most contextually relevant information for the final response.

These enhancements improve both recall and precision in the retrieval stage, ensuring that the generative model operates on high-quality context without incurring significant latency.

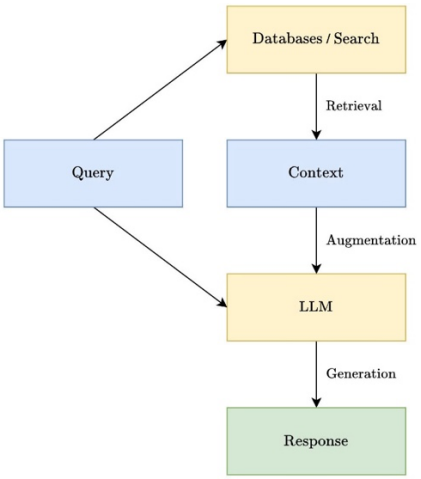


Figure 1. Architecture of a typical Retrieval-Augmented Generation (RAG) system.

IV. Methodology

In this section, we detail the individual components of our proposed method, outlining the specific roles and processes of each. After describing document embedding, AI, front end, and back end, we synthesize these components in the final subsection, demonstrating how they interact seamlessly through complete, practical use cases.

A. Document Embedding

In our system, all documents are ingested internally. Each document undergoes a standardized processing pipeline to ensure effective semantic search and retrieval. The main steps include:

- Step 1 - Format Conversion:

Uploaded documents in .docx or .pdf format are first converted to markdown (.md) format for easier processing by large language models (LLMs).

- Step 2 - Segmentation:

The markdown document is segmented by headers. If any segment exceeds a predefined maximum length, it is further split to ensure all segments are within the allowed token or character limit.

- Step 3 - Semantic Embedding:

Each segment is then embedded using a semantic embedding model to enable efficient and meaningful semantic search.

- Step 4 - Metadata Storage:

For each segment, metadata such as document title, page number, and position within the document is stored, supporting precise context tracking and flexible document retrieval.

B. Artificial Intelligence (AI)

In this section, we focus solely on the core AI logic and omit system scaling considerations. The core workflow consists of several sequential steps (also referred to as nodes in the pipeline): input preparation, information extraction, context retrieval, reranking, and answer generation & formatting.

1) Input Preparation

The process begins when a user query is received. The first AI node systematically constructs a distilled conversational memory for effective downstream processing. This involves:

- Step 1 – Recent Message Aggregation:

If the conversation is at an early stage and contains only a few messages, the current query is combined with all previous messages to preserve complete context.

- Step 2 – Memory Summarization:

When the conversation becomes lengthy, messages that are older than a certain threshold are summarized into a concise form known as memory, retaining important prior context while remaining within processing limits.

- Step 3 – Input Combination:

The input for the next step is constructed by combining the memory summary (if available), the most recent conversation messages, and the current user query.

For example, if the user asks, “Expand the last section of your answer,” the model references the last answer, recent conversation history, and summarized memory to generate the requested expansion with relevant details.

2) Information Extraction

In this step, the AI module extracts key information from the prepared memory and context: topic, task, and age. This process includes:

- Step 1 – LLM Invocation:

The system calls the large language model (LLM) and requests a structured JSON response with fields topic, task, and age.

- Step 2 – Response Processing:

The system interprets the extracted attributes to determine the workflow’s next action.

Handling of the extracted values proceeds as follows:

- Case 1 - Missing Age:

If the topic is provided but the age is missing, the system prompts the user to specify the age.

- Case 2 - Specified Topic and Age:

If both topic and age are specified, the system identifies the task. For certain predefined tasks (e.g., designing a lecture, expanding a section), a custom prompt is generated; otherwise, a general prompt is used.

- Case 3 - Unrelated Topic:

If the topic is unrelated to any internal document, the language model enters a general mode and acts as a standard conversational agent.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:46:29 UTC from IEEE Xplore. Restrictions apply.

A key challenge here is ensuring that the LLM prioritizes the most recent conversational information. For example, if the user switches topics midway, the system should detect and update the current topic, age, and task accordingly.

3) Context Retrieval

Once the key attributes have been determined, the system retrieves relevant contextual information to support an informed response. This node operates as follows:

- Step 1 – Topic Filtering:

Using the extracted topic, the system filters the database to select only documents that belong to the identified topic, retrieving all corresponding segments.

- Step 2 – Hybrid Retrieval:

Next, both keyword-based and semantic retrieval methods are applied to these filtered segments to identify the top k matches based on the current query and task.

The goal of this process is to ensure that the response generated in later steps is grounded in the most relevant and accurate information available for the user's needs.

4) Reanking

After retrieving potential knowledge segments, the system optimizes for relevance by reranking the candidates. This process includes:

- Step 1 – Relevance Scoring:

Each retrieved document segment or answer candidate is evaluated using a re-ranker model. This re-ranker is an LLM specifically designed and fine- tuned for assessing the contextual relevance of candidates with respect to the current query and task.

- Step 2 – Candidate Selection:

The top r candidates ($r < k$) with the highest relevance scores are selected for answer generation, ensuring that the response closely matches both the user's intent and the given context.

5) Answer Generation & Formatting

In the final step, the synthesized context and user query are provided to the main language model (LLM) to generate a draft answer. This output is then formatted to meet the requirements of both the identified task and the specified age.

This process includes:

- Step 1 – Answer Generation:

The main LLM is invoked with the constructed context and current query, producing a comprehensive answer aligned with the user's needs.

- Step 2 – Task-Based Formatting:

The response is formatted according to the outline, style, and requirements defined by the institution for the selected task.

- Step 3 – Age Appropriateness:

The language, terminology, and depth are further formatted to suit the age specified by the user, enhancing both clarity and accessibility.

C. Front End

The front end serves as the primary interface between users and the system. In this project, we emphasize simplicity and focus on showcasing the AI component by implementing only the core features required for a smooth workflow and clear demonstration, while keeping auxiliary features to a minimum. The architecture is inspired by platforms such as OpenAI's ChatGPT, featuring a straightforward registration/login page and a main chat session interface.

1) Authentication

The authentication system secures access and personalizes user interactions. It comprises the following stages:

- Sign-Up and Login: Users can create new accounts or log in using existing credentials through dedicated pages. All credential handling is performed securely via encrypted protocols.
- Session Management: Upon successful authentication, the front end securely stores session tokens and manages session states, enabling users to remain logged in across visits and devices unless they log out or their session expires.

2) Chat Interface

Once authenticated, users interact with the system's core functionality through an intuitive chat interface designed for efficient communication with the AI:

- Conversation Management: Users can start new conversations, revisit, or delete existing ones. Conversation histories are persistently stored and easily accessible via the interface.
- Query Submission: Users submit natural language queries, which are transmitted seamlessly to the back end for processing.
- AI Responses: AI-generated responses are displayed in a conversational format, enabling follow-up questions and contextual clarifications within the ongoing session.
- Prompt Suggestions: The interface offers context-aware prompt suggestions to assist users in formulating effective queries, improving efficiency and satisfaction.

D. Back End

The back end is responsible for managing all system functionalities beyond the core AI processing. Its primary roles include authenticating users, handling secure data storage and retrieval, and orchestrating workflows related to AI interactions. When a user submits an AI-related query, the back end forwards the request to the AI component and coordinates all necessary data exchanges, acting as the central hub for operational logic.

1) User Management

User management ensures secure access control and session handling. This process includes:

- Registration and Authentication: The back end securely processes user sign-up and login information, validating credentials and issuing authentication tokens in accordance with industry best practices.
- Session Handling: User sessions are managed using these authentication tokens, with support for session expiration, renewal, and logout. This ensures both robust security and a seamless user experience.

2) Data Management

Reliable storage and retrieval of user and conversation data are crucial for personalized and context-aware AI interactions:

- Profile Storage:

User profiles, along with relevant metadata for personalization and access control, are securely stored and managed.

- Conversation History:

All conversations are persistently logged and efficiently indexed, enabling rapid retrieval and maintaining contextual continuity for the AI across multiple interactions and sessions. AI Service Orchestration When the API is related to an AI service, the backend acts as a for- warder—passing the input from the frontend to the AI service and returning the output

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:46:29 UTC from IEEE Xplore. Restrictions apply.

from the AI back to the frontend. Simultaneously, it also stores necessary information in the database.

E. All Workflows

This section presents an end-to-end illustration of the system's operation in real scenarios, highlighting how the front end, back end, and AI components interact across typical user journeys.

1) Admin Uploads New Learning Material

The process of uploading new learning materials is initiated by an admin directly on the internal system rather than via the front-end interface. The following workflow illustrates how files are processed, converted, and stored in the document database, with status updates provided throughout.

- The admin initiates a file upload directly on the Database of AI then execute Python code to trigger the process.
- There is an separate AI pipeline for processing documents, and stores the resulting data into the document database, which only require user to upload document to the right folder and document are in supported format.
- The system display the progress through terminal.

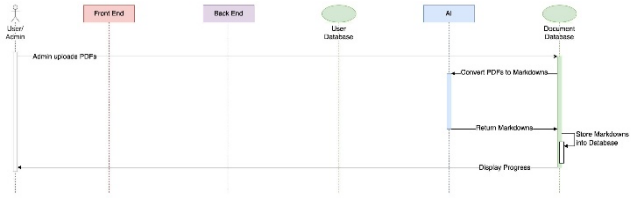


Figure 2. Workflow illustrating admin uploads of PDFs and processing into the database

2) User Sign Up

The registration workflow guides new users through entering their information, submitting it, and receiving system feedback. This enables creation of a secure and validated user record for subsequent access.

- The user interacts with the sign-up page to enter registration details.
- The front end forwards the information to the back end, which validates and stores the data in the user database.
- Registration confirmation (or error) is communicated back to the user.

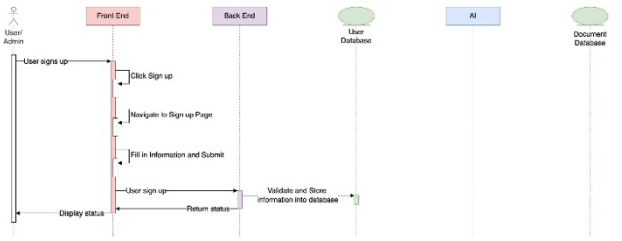


Figure 3. User workflow for account registration and data validation.

3) User Login

Upon logging in, users authenticate with the front end and are validated by the back end. This workflow covers credential submission, session management, and post-login navigation.

- Users submit credentials via the front end.
- The back end verifies information against the user database and manages session tokens.
- Upon success, the user is redirected to the chat page.

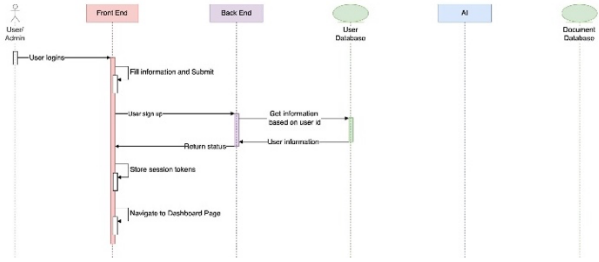


Figure 4. User login process, verification, and session management

4) Get Conversations

When a user logged in and access their chat web page, the system automatically retrieve all user past conversation data and display it for them to select past conversation. To optimize this we separate it into two phrases, first we only fetch all conversation titles and then when user select specific conversation we will crawl that conversation data.

- The user accesses the chat webpage, which triggers the system to retrieve chat history based on the user ID.
- All conversation titles are fetched and displayed in the sidebar of the user's webpage.
- When the user clicks on a specific title, all messages associated with that title are fetched and displayed in the chat section.
- Workflow illustrating admin uploads of PDFs and processing into the database.

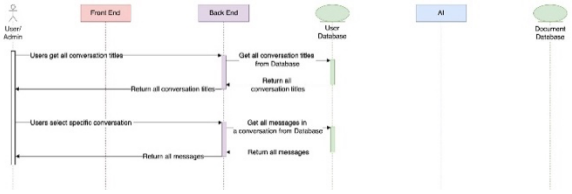


Figure 5. Retrieving all historical conversations for an admin user

5) Conversation Creation

Users can begin new discussions by creating conversations, which are processed and registered within the system for ongoing interaction. The following workflow shows this process, including title assignment.

- Users initiate a new conversation through the front end.
- The back end creates the conversation, stores it in the user database, and confirms creation.
- Additional metadata, such as conversation title, may be extracted and updated automatically when the AI gets the overall content of the conversation.

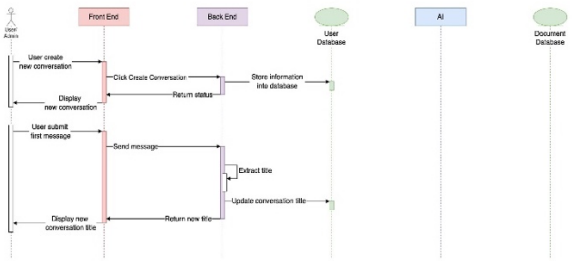


Figure 6. Workflow for creating a new conversation and automatic title extraction.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:46:29 UTC from IEEE Xplore. Restrictions apply.

6) Delete Conversation

To support data management and privacy, users may delete conversations. This workflow details the sequence from deletion request to database update and removal confirmation.

- Users choose to delete a conversation from the user interface.
- The conversation is removed from the system following confirmation by the back end and user database update.

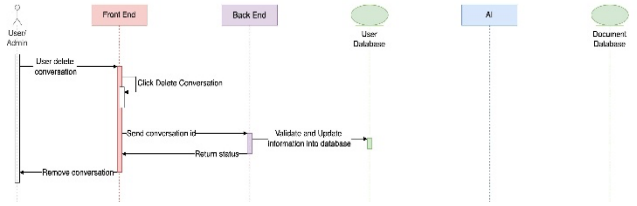


Figure 7. User workflow for deleting a conversation and confirming removal

7) Chat

The chat feature leverages AI and historical data to enable context-aware and responsive conversations. The workflow below illustrates how user queries are processed to provide intelligent system responses.

- Users submit messages in a conversation.
- The front end sends all conversation messages to the back end, which formats the query for the AI component.
- The AI extracts relevant information and context, queries the database if necessary, and returns a response.
- The response is displayed to the user and stored in the database.

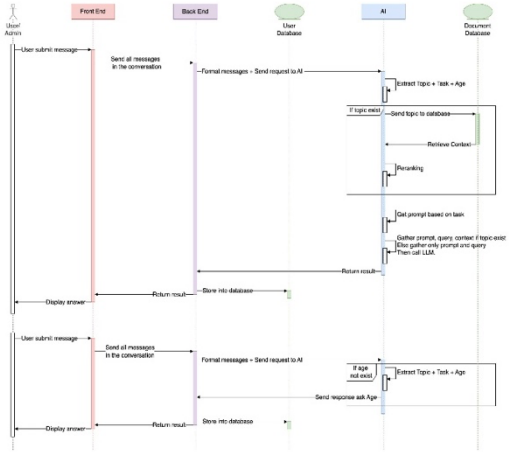


Figure 8. Multi-step chat workflow, with context extraction and AI-powered answer generation

8) Get All Conversations

Admin can access all conversation history with username encrypted, which can be use for later evaluation by interact directly with backend.

- Admin submit request to backend server.
- The server return all conversation history in JSON format.

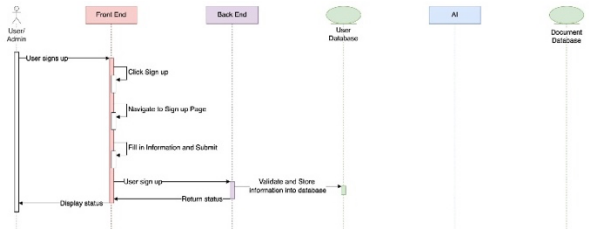


Figure 9. Admin workflow for retrieving all conversations for later analysis

V. Experiment

A. Implementation Details

This subsection provides technical specifics regarding the components and technologies used in our system, including language models, storage solutions, and the frameworks supporting the front and back end. Specifically, we use qwen2.5:14b for both embedding generation and as the core large language model, while mxbai-rerank-large-v2 is employed for reranking. Embeddings are stored and queried using ChromaDB, our vector database. For data preprocessing and orchestration, we leverage the MinerU package for document processing, Langchain for chunking, and LangGraph as the orchestrator. The front end is built with React.js to provide an interactive user interface. On the back end, we use FastAPI to serve APIs, with JWT for authentication and SQLModel for database interactions.

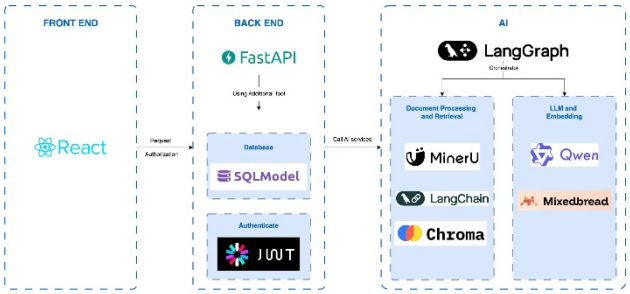


Figure 10. Overview of tools and tech stack of our pipeline

B. User Interface Overview

To illustrate the usability and design of our system, this subsection presents the graphical interface that end users interact with. The web-based frontend, developed using React.js, provides a seamless experience across various functionalities including account creation, authentication, and real-time interaction with the chat system powered by our backend language models.

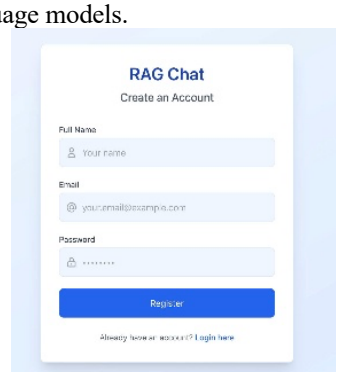


Figure 11. Signup page allowing new users to register by providing basic information.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:46:29 UTC from IEEE Xplore. Restrictions apply.

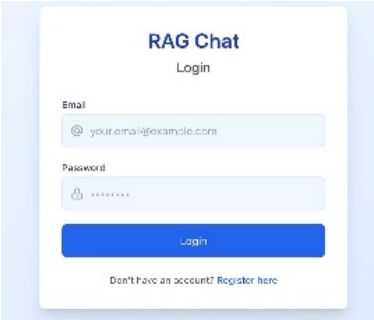


Figure 12. Login page for user authentication using JWT

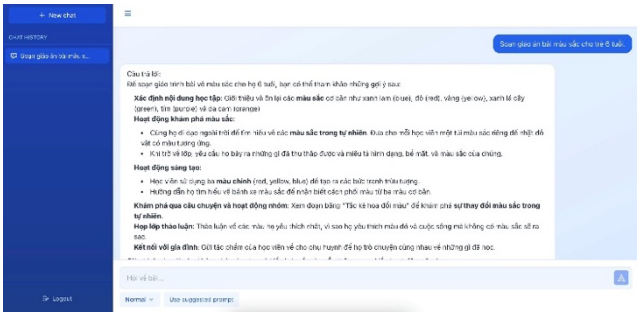


Figure 13. Chat interface showcasing interactions with the language model

VI. Conclusion

In this work, we have presented a customized AI chatbot designed specifically for educational environments, advancing beyond the limitations of general-purpose large language models. Our system uniquely integrates rule-based filtering, retrieval-augmented generation (RAG), and user-adaptive response strategies, drawing upon curated, lesson-aligned internal resources to deliver accurate, contextually relevant, and pedagogically consistent answers. By combining hybrid retrieval methods and contextual reranking, the chatbot is able to produce responses tailored to user attributes such as age group and specific educational tasks. Leveraging a high-performing and compact open-source model (Qwen-14B) enables on-premises deployment on modest hardware, reducing operational costs and ensuring data privacy—key requirements for educational institutions. The robust workflow, which encompasses advanced document processing, sophisticated AI orchestration, and seamless integration between the front end and back end, demonstrates the system's ability to operate effectively in real-world settings.

For future work, further opportunities lie in enhancing personalization through deeper user modeling, expanding the range of supported educational content, and scaling the system for use in larger or multi-lingual institutions. Overall, our method provides an effective, adaptable, and institutionally aligned conversational AI framework, contributing to the evolving landscape of educational technology.

Acknowledgment

This research is financially supported by Google Academic Research Award 2024 project: Empowering Early Childhood Education (ECE) Teachers with AI: Enhancing Professional Growth through Personalized Support and Timely Feedback.

This research was supported by RMIT AWS Cloud Supercomputing (RACE).

References

[1] OpenAI *et al.*, "GPT-4 Technical Report," Mar. 04, 2024, *arXiv*: arXiv:2303.08774. doi: 10.48550/arXiv.2303.08774.

[2] Z. Zhang and X. Huang, "The impact of chatbots based on large language models on second language vocabulary acquisition," *Heliyon*, vol. 10, no. 3, p. e25370, Feb. 2024, doi: 10.1016/j.heliyon.2024.e25370.

[3] "Gemini," Google DeepMind. Accessed: Sep. 13, 2025. [Online]. Available: https://deepmind.google/models/gemini/

[4] M. Ozkaya, "LLM Models: OpenAI ChatGPT, Meta LLaMA, Anthropic Claude, Google Gemini, Mistral AI, and xAI Grok," Medium. Accessed: Sep. 13, 2025. [Online]. Available: https://mehmetozkaya.medium.com/llm-models-openai-chatgpt-meta-llama-anthropic-claude-google-gemini-mistral-ai-and-xai-grok-bd35779704c2

[5] "Overview," Perplexity. Accessed: Sep. 13, 2025. [Online]. Available: https://docs.perplexity.ai/getting-started/overview

[6] D. Ifenthaler *et al.*, "Artificial Intelligence in Education: Implications for Policymakers, Researchers, and Practitioners," *Technol. Knowl. Learn.*, vol. 29, no. 4, pp. 1693–1710, Dec. 2024, doi: 10.1007/s10758-024-09747-0.

[7] J. A. Delello, W. Sung, K. Mokhtari, J. Hebert, A. Bronson, and T. De Giuseppe, "AI in the Classroom: Insights from Educators on Usage, Challenges, and Mental Health," *Educ. Sci.*, vol. 15, no. 2, p. 113, Feb. 2025, doi: 10.3390/educsci15020113.

[8] J. Wang and W. Fan, "The effect of ChatGPT on students' learning performance, learning perception, and higher-order thinking: insights from a meta-analysis," *Humanit. Soc. Sci. Commun.*, vol. 12, no. 1, p. 621, May 2025, doi: 10.1057/s41599-025-04787-y.

[9] *QwenLM/Qwen*. (Sep. 13, 2025). Python. Qwen. Accessed: Sep. 13, 2025. [Online]. Available: https://github.com/QwenLM/Qwen

[10] A. T. Neumann, Y. Yin, S. Sowe, S. Decker, and M. Jarke, "An LLM-Driven Chatbot in Higher Education for Databases and Information Systems," *IEEE Trans. Educ.*, vol. 68, no. 1, pp. 103–116, Feb. 2025, doi: 10.1109/TE.2024.3467912.

[11] E. Voultsiou and L. Moussiades, "A systematic review of AI, VR, and LLM applications in special education: Opportunities, challenges, and future directions," *Educ. Inf. Technol.*, vol. 30, no. 13, pp. 19141–19181, Aug. 2025, doi: 10.1007/s10639-025-13550-4.

[12] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Comput Surv*, vol. 55, no. 12, p. 248:1-248:38, Mar. 2023, doi: 10.1145/3571730.

[13] O. Ali, P. A. Murray, M. Momin, Y. K. Dwivedi, and T. Malik, "The effects of artificial intelligence applications in educational settings: Challenges and strategies," *Technol. Forecast. Soc. Change*, vol. 199, p. 123076, Feb. 2024, doi: 10.1016/j.techfore.2023.123076.

[14] S. Feng, W. Shi, Y. Wang, W. Ding, V. Balachandran, and Y. Tsvetkov, "Don't Hallucinate, Abstain: Identifying LLM Knowledge Gaps via Multi-LLM Collaboration," Jul. 01, 2024, *arXiv*: arXiv:2402.00367. doi: 10.48550/arXiv.2402.00367.

[15] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, in NIPS '20. Red Hook, NY, USA: Curran Associates Inc., Dec. 2020, pp. 9459–9474.

[16] Z. Li, Z. Wang, W. Wang, K. Hung, H. Xie, and F. L. Wang, "Retrieval-augmented generation for educational application: A systematic survey," *Comput. Educ. Artif. Intell.*, vol. 8, p. 100417, Jun. 2025, doi: 10.1016/j.caeai.2025.100417.

[17] Z. Levonian *et al.*, "Retrieval-augmented Generation to Improve Math Question-Answering: Trade-offs Between Groundedness and Human Preference," Nov. 11, 2023, *arXiv*: arXiv:2310.03184. doi: 10.48550/arXiv.2310.03184.

[18] J. Rao and J. Lin, "RAMO: Retrieval-Augmented Generation for Enhancing MOOCs Recommendations," Jul. 06, 2024, *arXiv*: arXiv:2407.04925. doi: 10.48550/arXiv.2407.04925.

[19] H. Dinh and T. K. Tran, "EduChat: An AI-Based Chatbot for University-Related Information Using a Hybrid Approach," *Appl. Sci.*, vol. 13, no. 22, p. 12446, Jan. 2023, doi: 10.3390/app132212446.

[20] L. Yan *et al.*, "VizChat: Enhancing Learning Analytics Dashboards with Contextualised Explanations Using Multimodal Generative AI Chatbots," in *Artificial Intelligence in Education - 25th International Conference, AIED 2024 Recife, Brazil, July 8–12, 2024 Proceedings, Part II*, Springer, 2024, pp. 180–193. doi: 10.1007/978-3-031-64299-9_13.

[21] DeepSeek-AI *et al.*, "DeepSeek-V3 Technical Report," Feb. 18, 2025, *arXiv*: arXiv:2412.19437. doi: 10.48550/arXiv.2412.19437.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:46:29 UTC from IEEE Xplore. Restrictions apply.